

CIS 5357

Computing for Data Analytics

Week 14

18 APRIL 2024

Quick review on aggregating

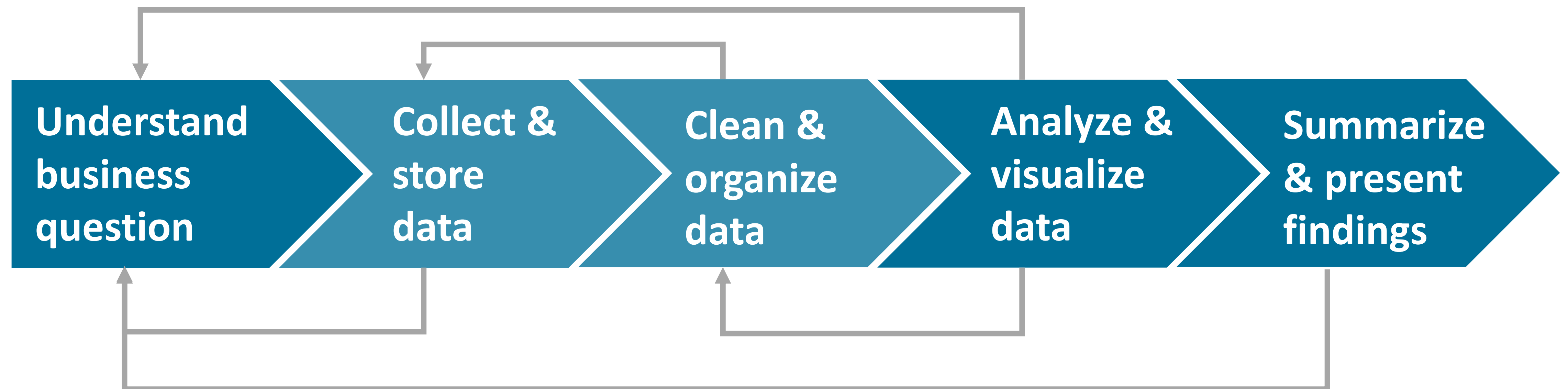
- return the number of rows including NaN missing values `size()`
- return the number of rows not including NaN missing values `count()`
- return the number of unique values, ignoring any NaN missing values `nunique()`
- create groups over more than one column, for the same measure `groupby()`
- create groups over more than one column, for different measures `agg()`
- create data crosstabs `unstack()`
- create groups for datetime columns `dt.function` or `resample()`

This week

Apply learnings to Dig datasets + Final Exam

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



Dig orders dataframe from Chapter 7

df_orders

ORDER_ID
DATETIME
RESTAURANT_ID
TYPE
DRINKS
COOKIES
MAIN
BASE
SIDE_1
SIDE_2
RESTAURANT_NAME
MAIN_NAME
BASE_NAME
SIDE_1_NAME
SIDE_2_NAME

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
✓ 0.0s
```

```
df_orders = pd.read_pickle('Chapter 7/orders.pickle')  
✓ 0.7s
```

Q1: where to launch drinks?

What if Dig decides to **launch a new line of drinks of its own**?
They want to launch **at a single location first**, then expand.

How might Dig use its data to decide at **which restaurant to launch** this new product line, and **when in the year** to do it?

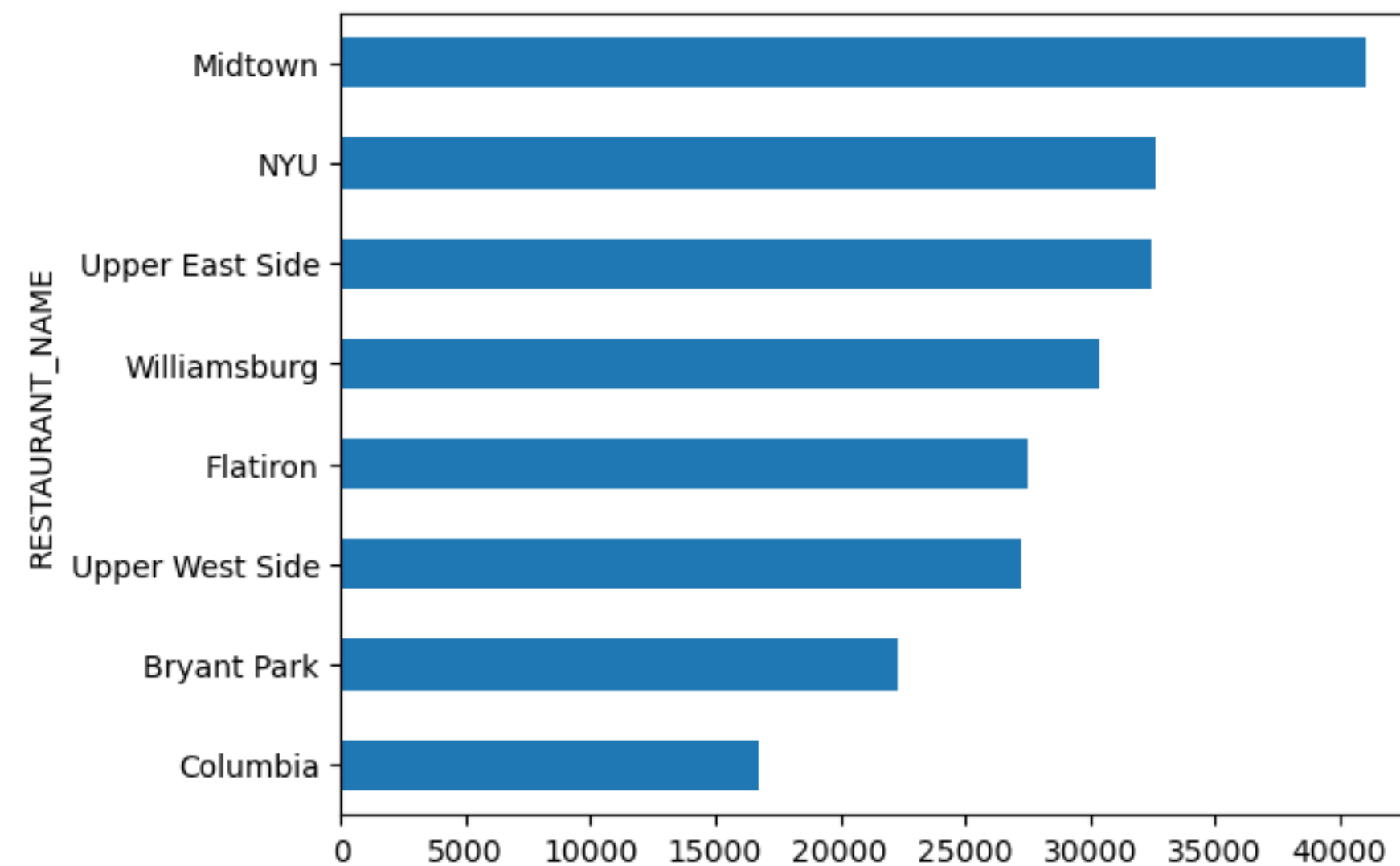
Where are drinks popular?

```
df_orders.groupby('RESTAURANT_NAME').DRINKS.sum().sort_values().plot(kind='barh')  
plt.title("Total drinks per location", loc="left", pad=50, fontsize=20)
```

✓ 0.2s

Text(0.0, 1.0, 'Total drinks per location')

Total drinks per location

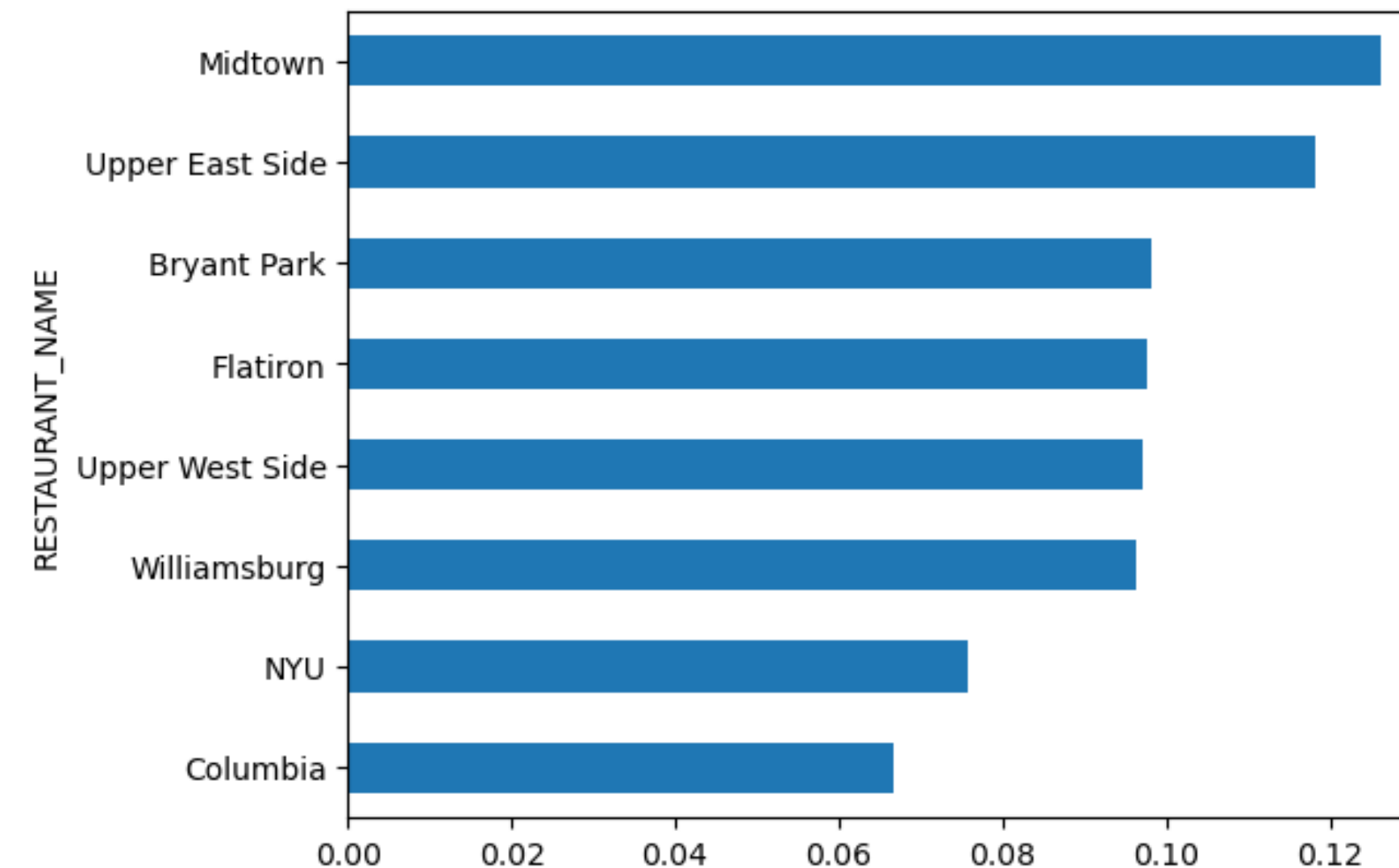


```
df_orders.groupby('RESTAURANT_NAME').DRINKS.mean().sort_values().plot(kind='barh')  
plt.title("Average drinks per location", loc="left", pad=50, fontsize=20)
```

✓ 0.1s

Text(0.0, 1.0, 'Average drinks per location')

Average drinks per location



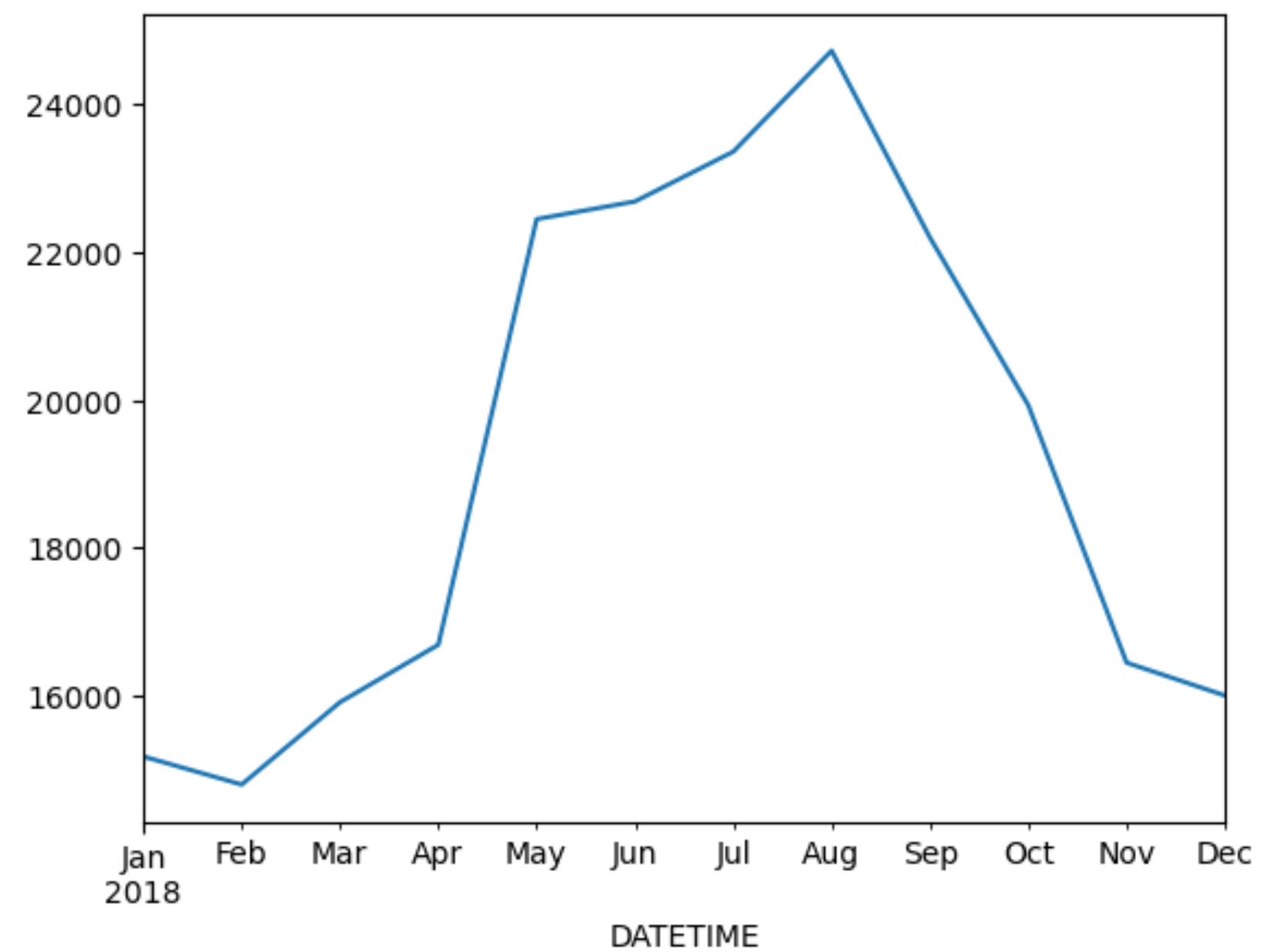
When should we launch?

```
df_orders.set_index('DATETIME').resample('M').DRINKS.sum().plot()  
plt.title("Drink orders by month", loc="left", pad=50, fontsize=20)
```

✓ 2.9s

Text(0.0, 1.0, 'Drink orders by month')

Drink orders by month



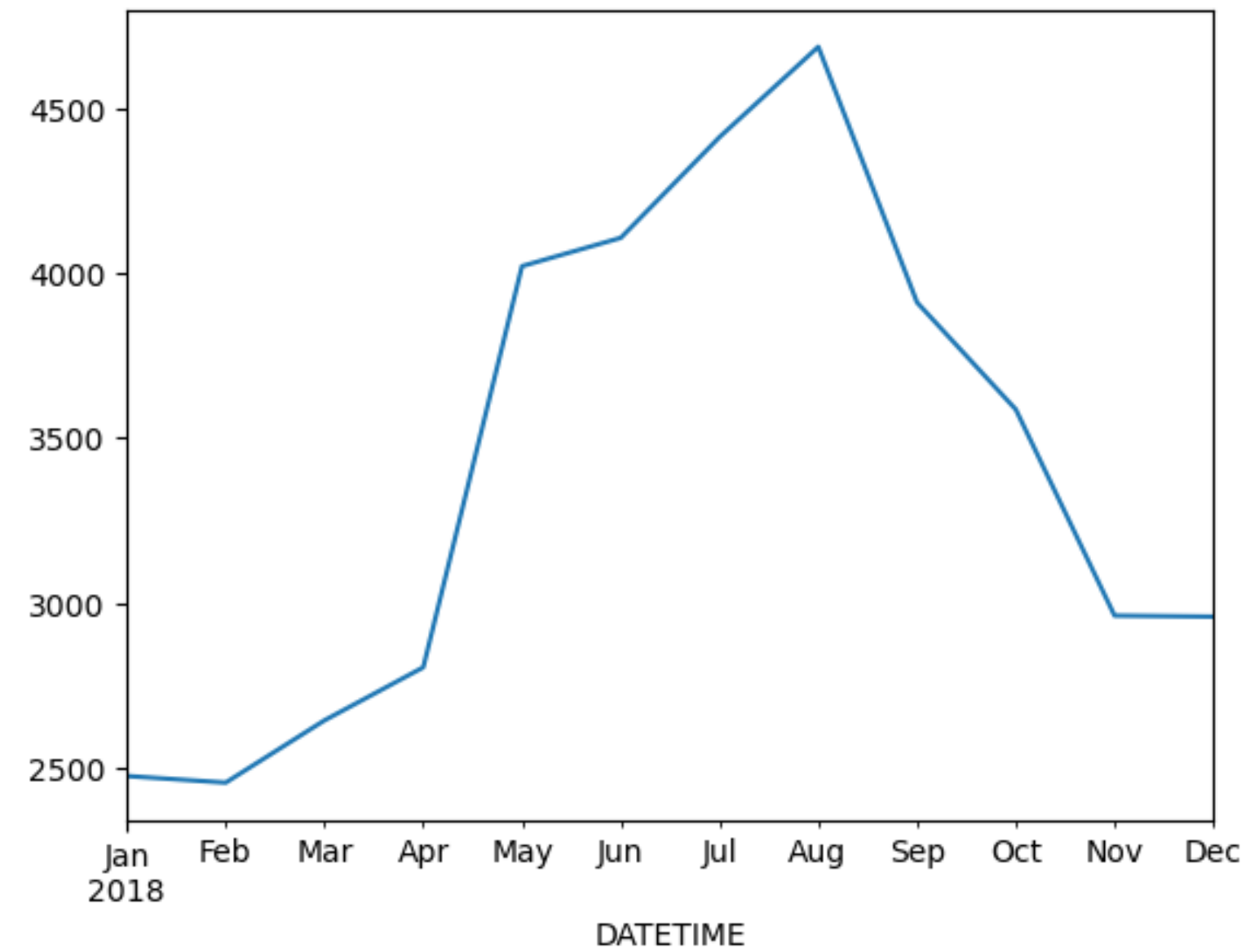
For Midtown specifically?

```
df_orders[df_orders.RESTAURANT_NAME == 'Midtown'].set_index('DATETIME').resample('M').DRINKS.sum().plot()  
plt.title("MIDTOWN drink orders by month", loc="left", pad=50, fontsize=20)
```

✓ 0.4s

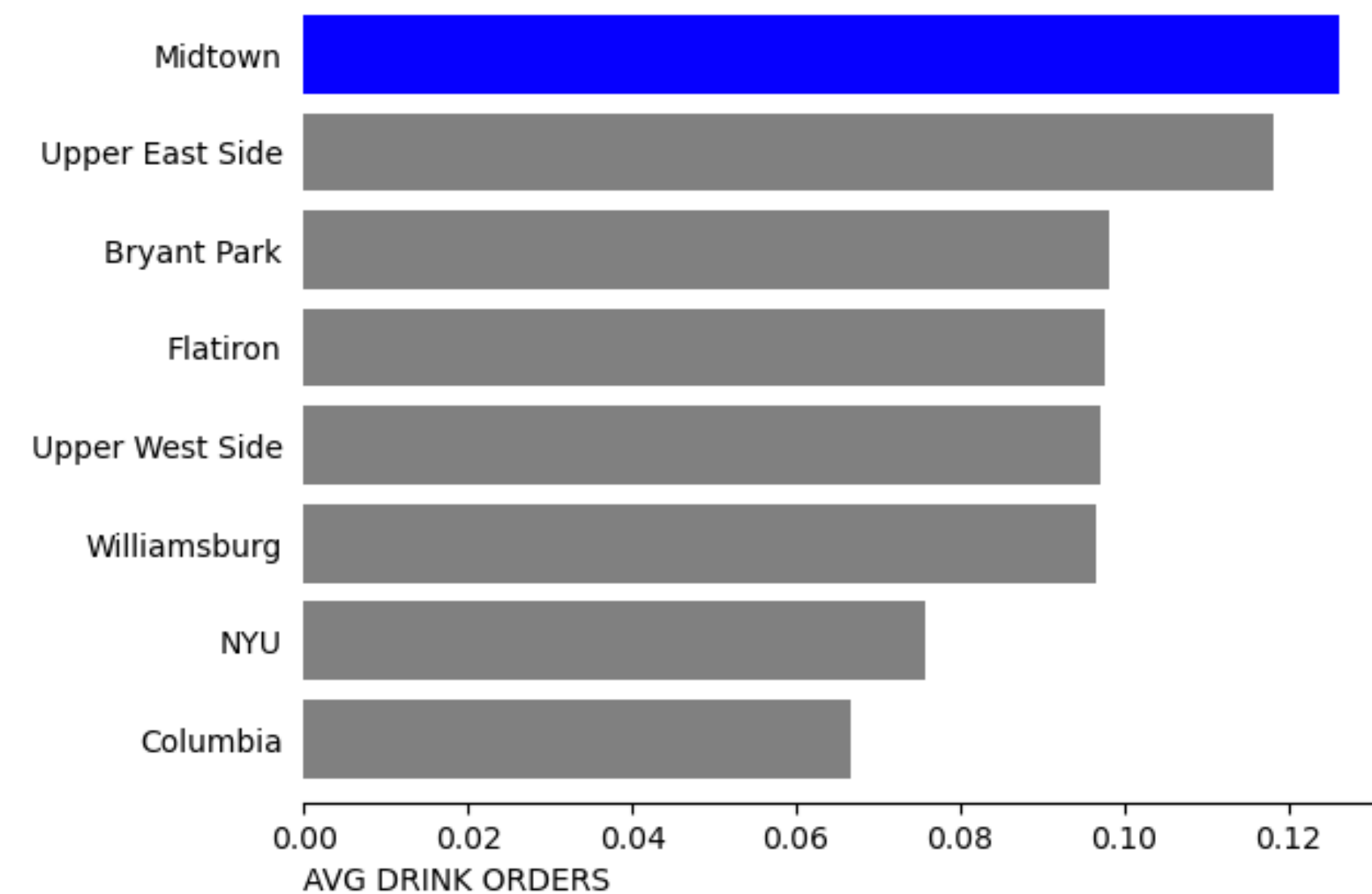
Text(0.0, 1.0, 'MIDTOWN drink orders by month')

MIDTOWN drink orders by month

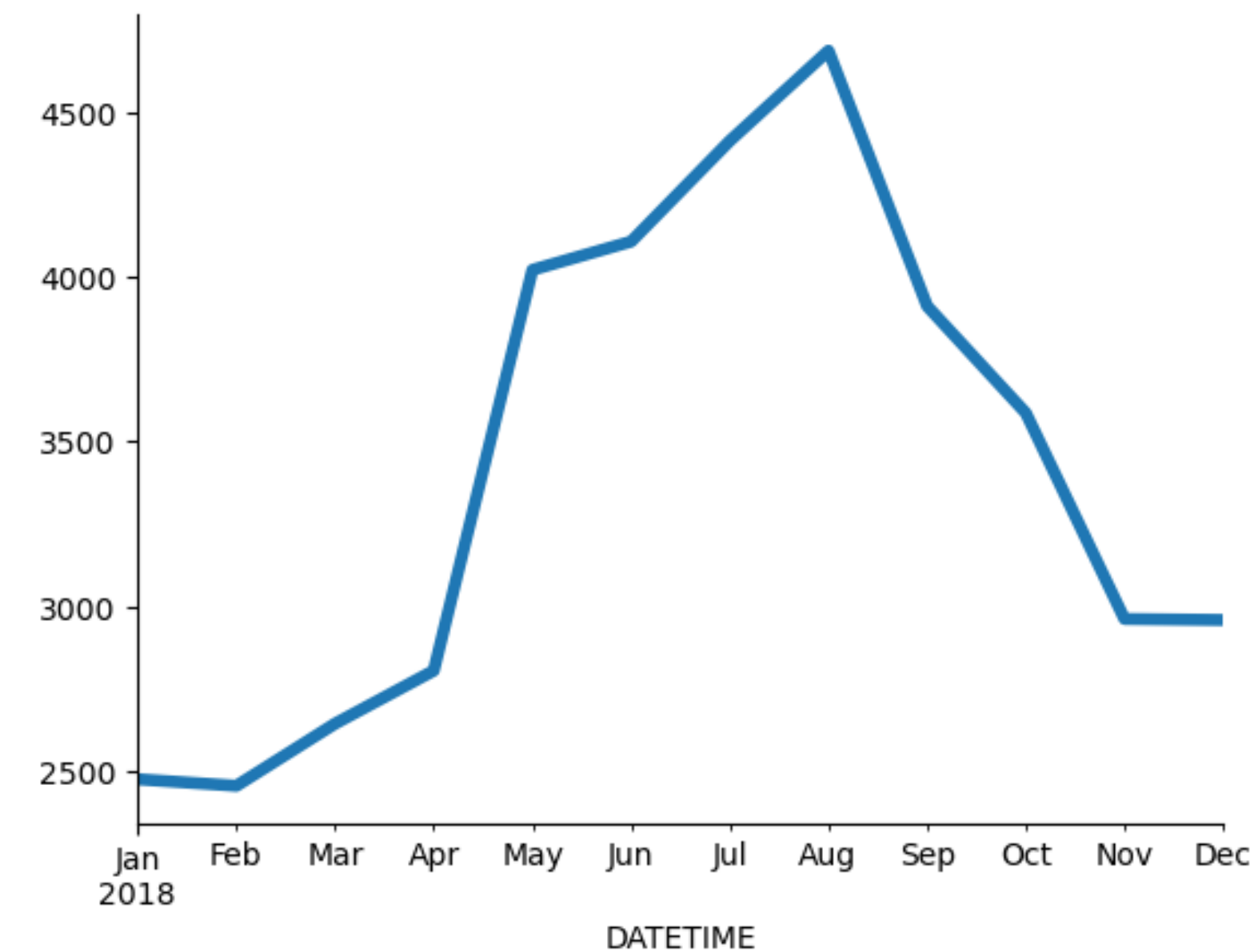


Let's launch drinks in Midtown this summer

MIDTOWN has the highest average drinks



Most of MIDTOWN drink orders occur in the summer



Q2: what to include on delivery menu?

What if Dig decides to create a brand-new, reimagined **delivery service**, with an entirely **new menu** and platform, built and optimized specifically for delivery.

Let's discuss questions Dig might answer using its data to support this effort.

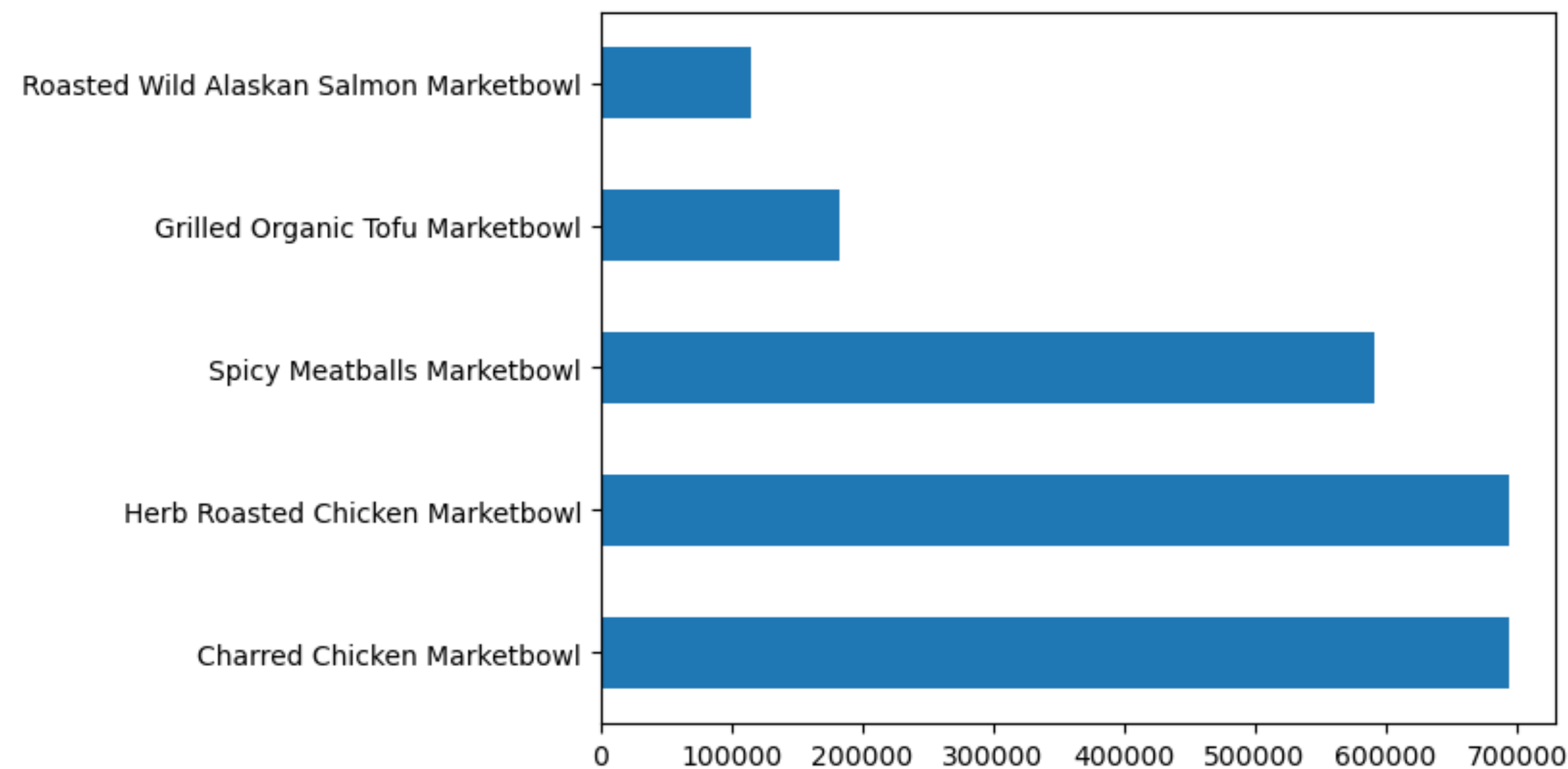
What's the most popular main?

```
df_orders.MAIN_NAME.value_counts().plot(kind='barh')  
plt.title("Count of orders by main", loc="left", pad=30, fontsize=20)
```

✓ 0.0s

Text(0.0, 1.0, 'Count of orders by main')

Count of orders by main



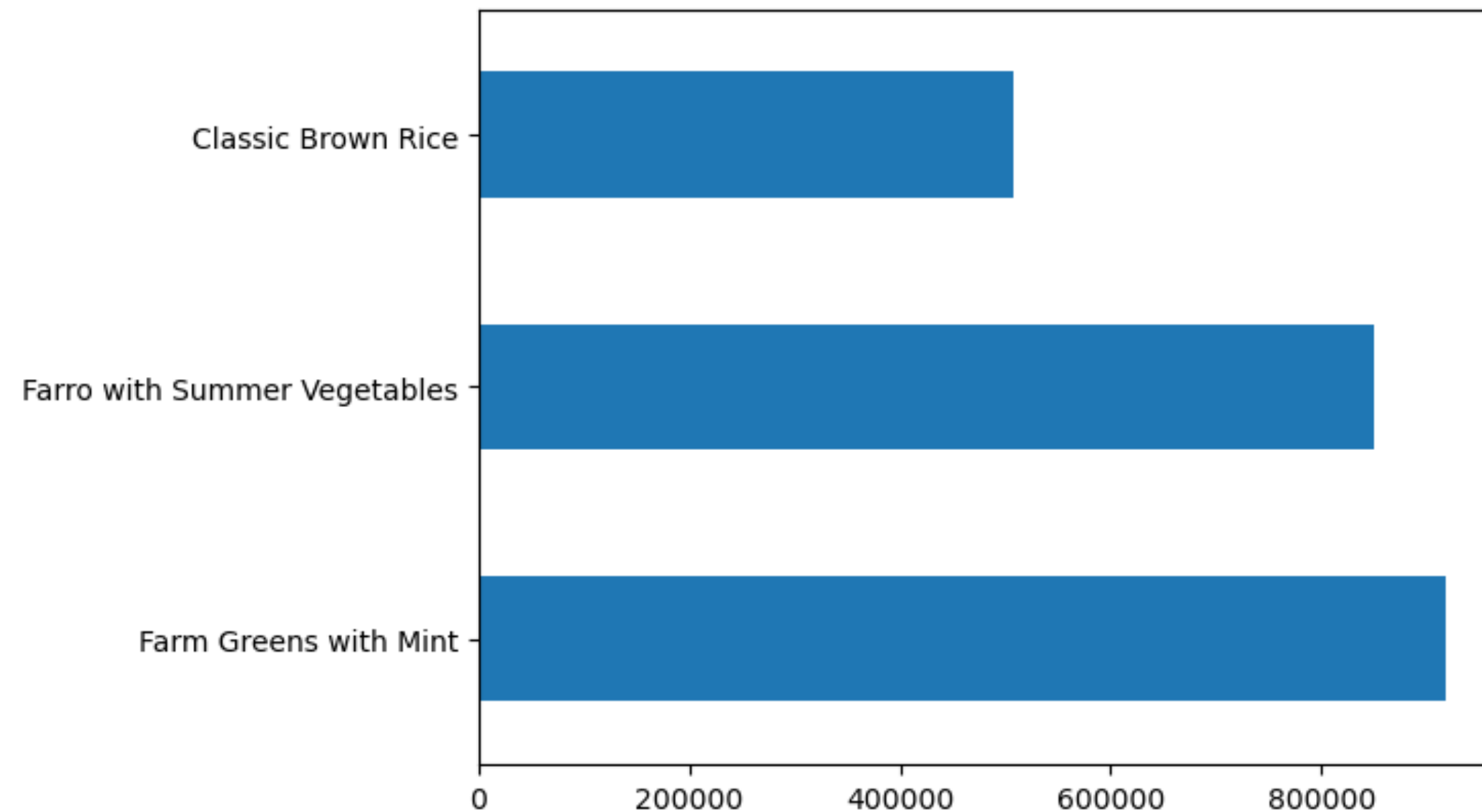
What's the most popular base?

```
df_orders.BASE_NAME.value_counts().plot(kind='barh')  
plt.title("Count of orders by base", loc="left", pad=30, fontsize=20)
```

✓ 0.0s

Text(0.0, 1.0, 'Count of orders by base')

Count of orders by base



Most popular main with which base?

```
df_orders.groupby(['MAIN_NAME', 'BASE_NAME']).ORDER_ID.count().unstack()
```

✓ 0.2s

BASE_NAME	Classic Brown Rice	Farm Greens with Mint	Farro with Summer Vegetables
MAIN_NAME			
Charred Chicken Marketbowl	155311	279591	258945
Grilled Organic Tofu Marketbowl	41323	73682	68153
Herb Roasted Chicken Marketbowl	154203	280243	259056
Roasted Wild Alaskan Salmon Marketbowl	25616	46052	42779
Spicy Meatballs Marketbowl	132060	238509	220116

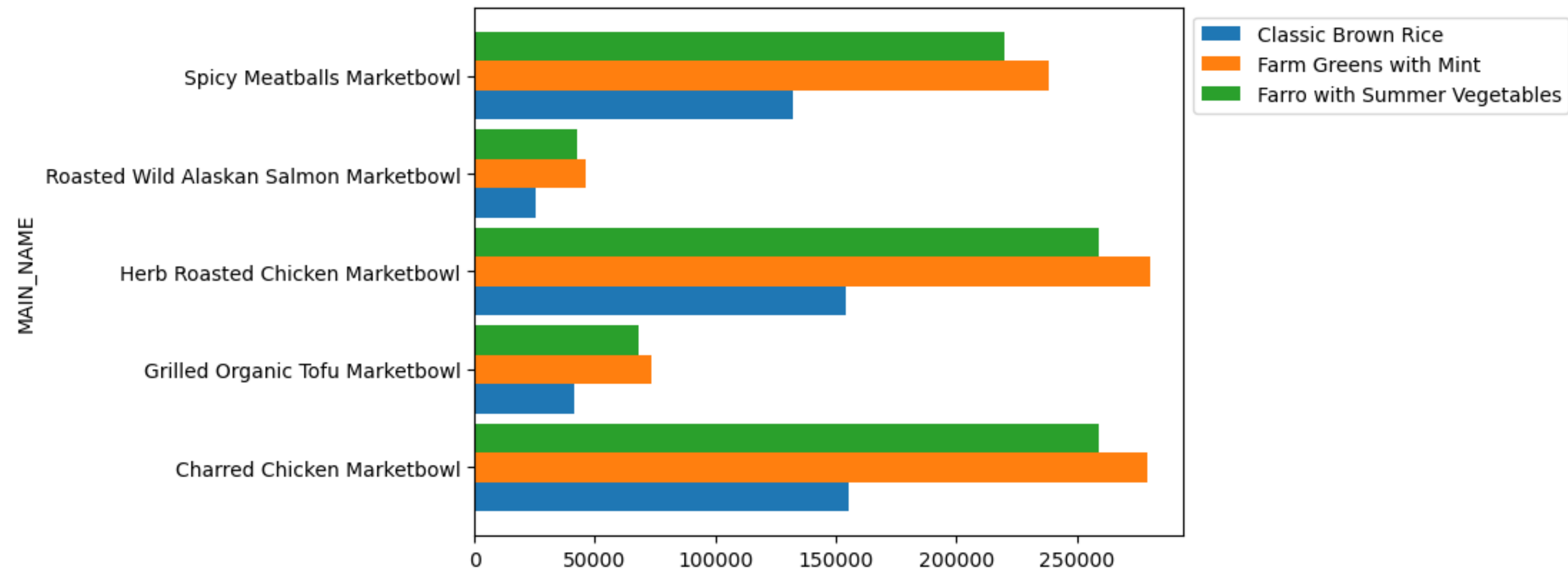
Let's visualize mains with bases!

```
df_orders.groupby(['MAIN_NAME', 'BASE_NAME']).ORDER_ID.count().unstack().plot(kind='barh', width = 0.9).legend(bbox_to_anchor=(1, 1))  
plt.title("Orders by main and base", loc="left", pad=30, fontsize=20)
```

✓ 0.3s

Text(0.0, 1.0, 'Orders by main and base')

Orders by main and base



Challenge: how about sides?

What are the most popular sides with each main?

Challenge: how about sides?

What are the most popular sides with each main?

We'll resume in
5 minutes...

Challenge: how about sides?

What are the most popular sides with each main?

We'll resume in
4 minutes...

Challenge: how about sides?

What are the most popular sides with each main?

We'll resume in
3 minutes...

Challenge: how about sides?

What are the most popular sides with each main?

We'll resume in
2 minutes...

Challenge: how about sides?

What are the most popular sides with each main?

We'll resume in
1 minute...

Challenge: how about sides?

What are the most popular sides with each main?

Challenge: how about sides?

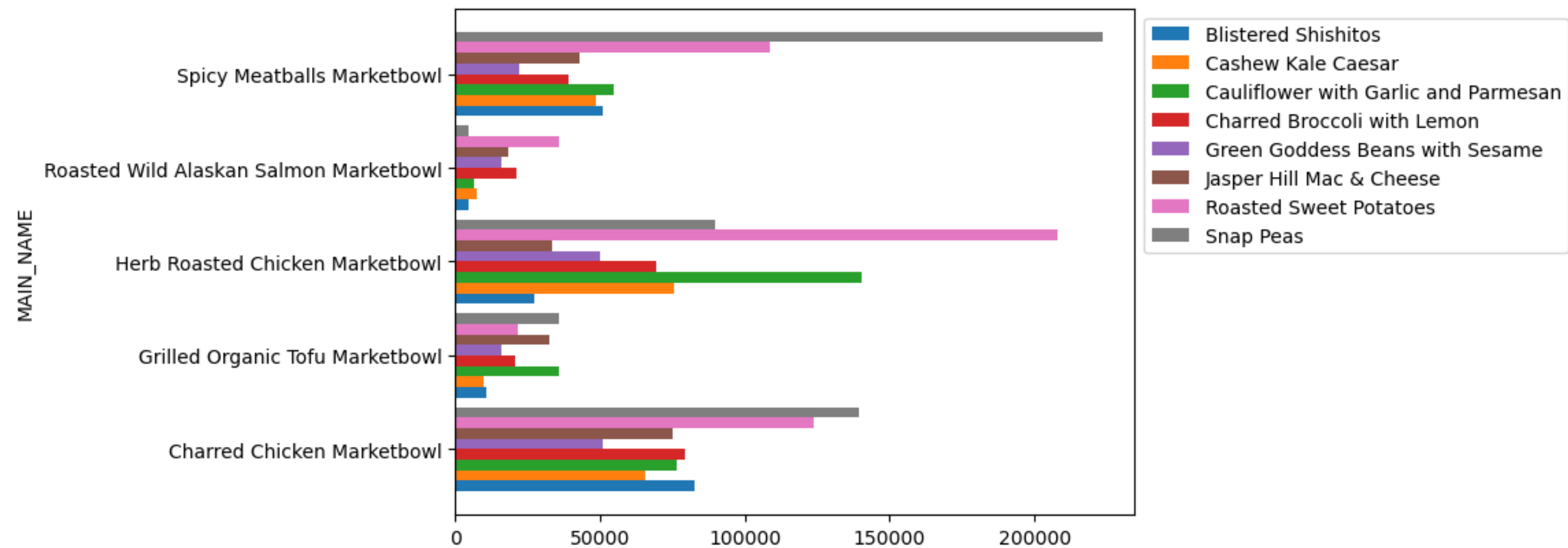
What are the most popular sides with each main?

```
df_orders.groupby(['MAIN_NAME', 'SIDE_1_NAME']).ORDER_ID.count().unstack().plot(kind='barh', width = 0.9).legend(bbox_to_anchor=(1, 1))
plt.title("Orders by main and side 1", loc="left", pad=30, fontsize=20)
```

✓ 0.4s

Text(0.0, 1.0, 'Orders by main and side 1')

Orders by main and side 1



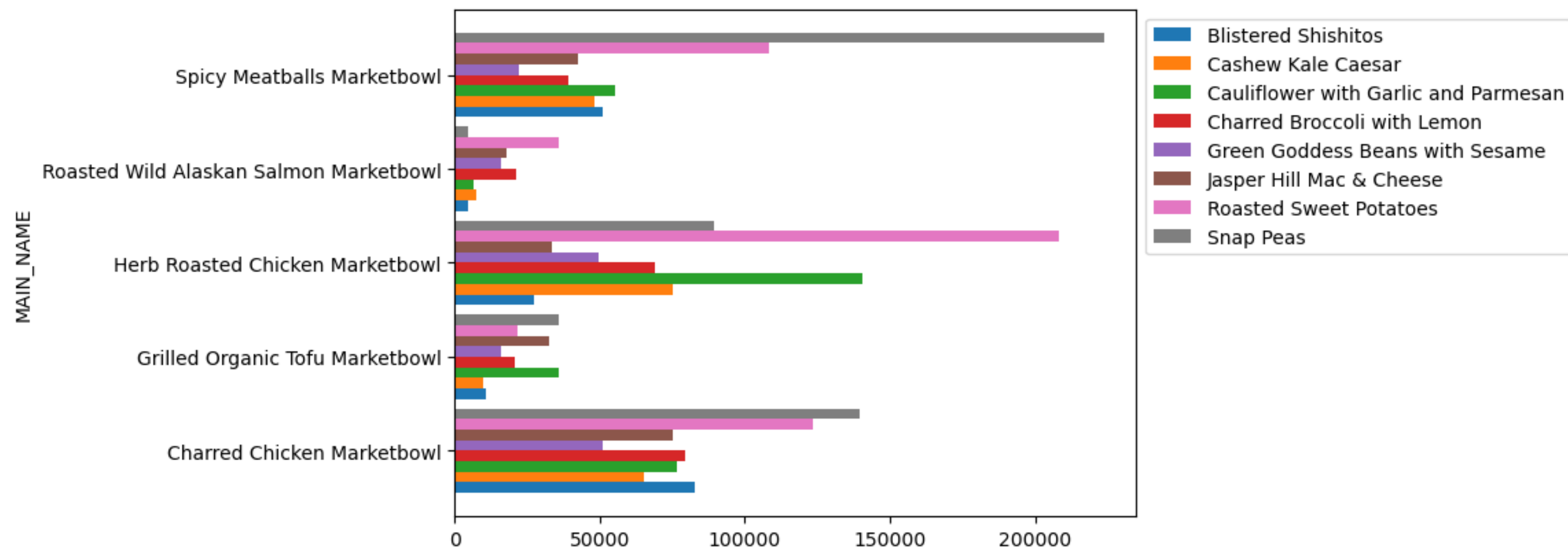
Side 2 looks very similar

```
df_orders.groupby(['MAIN_NAME', 'SIDE_2_NAME']).ORDER_ID.count().unstack().plot(kind='barh', width=0.9).legend(bbox_to_anchor=(1, 1))
plt.title("Orders by main and side 2", loc="left", pad=30, fontsize=20)
```

✓ 0.3s

Text(0.0, 1.0, 'Orders by main and side 2')

Orders by main and side 2



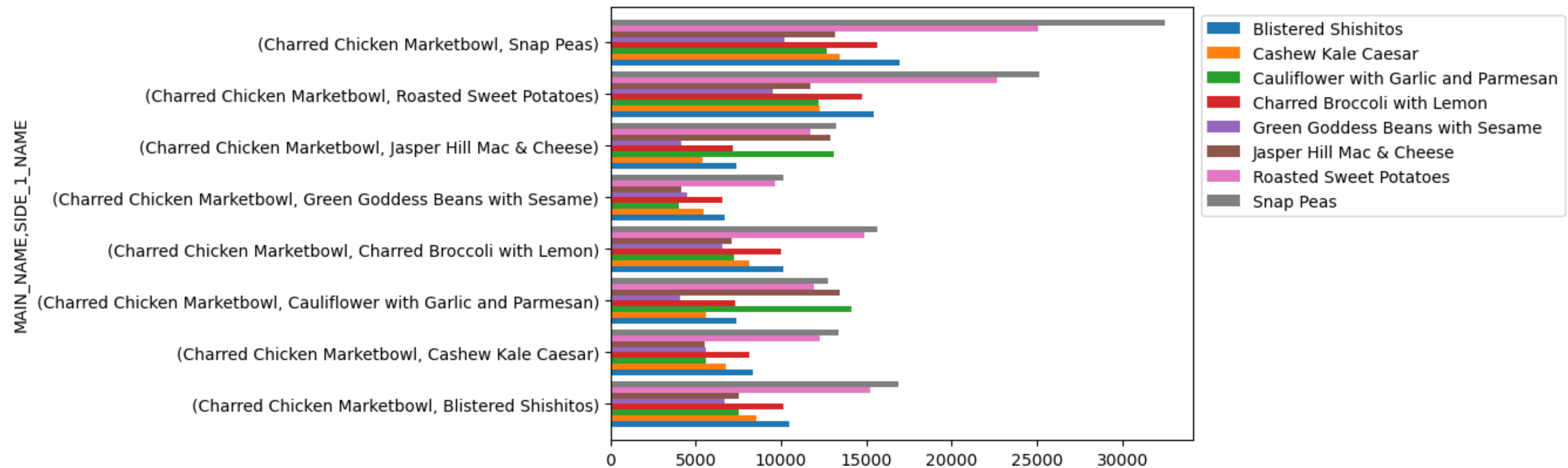
Some double up on sides

```
df_orders[df_orders.MAIN_NAME == 'Charred Chicken Marketbowl'].groupby(['MAIN_NAME', 'SIDE_1_NAME', 'SIDE_2_NAME']).ORDER_ID.count().unstack().plot(kind='barh', width = 0.9).legend(bbox_to_anchor=(1, 1))  
plt.title("Charred Chicken with sides", loc="left", pad=30, fontsize=20)
```

✓ 0.7s

Text(0.0, 1.0, 'Orders by main and side 2')

Orders by main and side 2



Let's take a quick break

We'll resume in

8 minutes...

Let's take a quick break

We'll resume in

7 minutes...

Let's take a quick break

We'll resume in

6 minutes...

Let's take a quick break

We'll resume in

5 minutes...

Let's take a quick break

We'll resume in

4 minutes...

Let's take a quick break

We'll resume in

3 minutes...

Let's take a quick break

We'll resume in
2 minutes...

Let's take a quick break

We'll resume in

1 minute...

Q3: where to expand delivery service?

Where should we pilot a **new delivery menu**?

How could we use the orders information to decide which restaurant to use as a pilot for the new program?

Where are deliveries popular?

```
df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100
```

✓ 0.6s

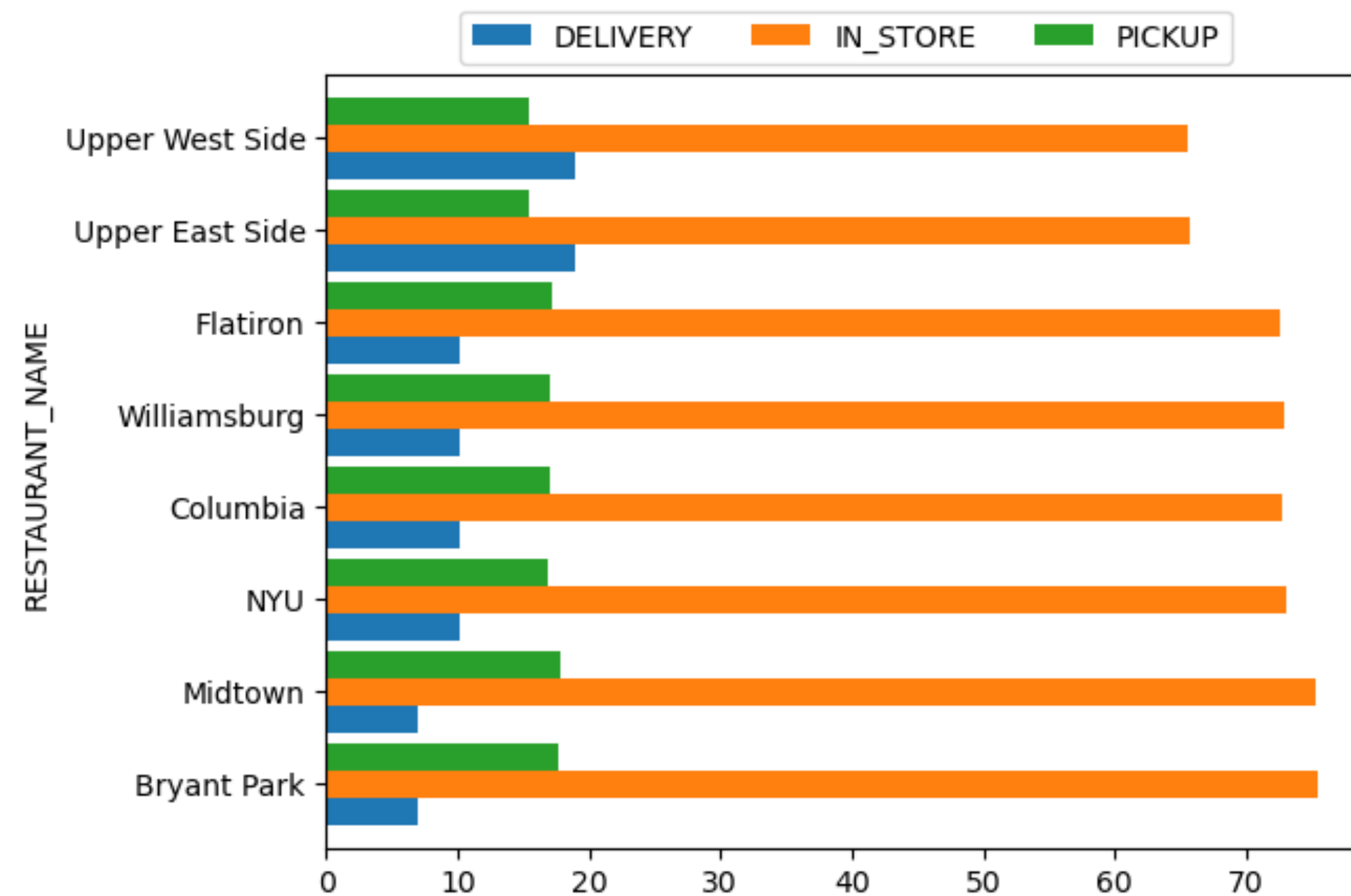
	TYPE	DELIVERY	IN_STORE	PICKUP
RESTAURANT_NAME				
	Bryant Park	6.9	75.4	17.7
	Midtown	6.9	75.3	17.8
	NYU	10.1	73.1	16.9
	Columbia	10.1	72.8	17.1
	Williamsburg	10.1	72.9	17.0
	Flatiron	10.2	72.6	17.2
	Upper East Side	18.9	65.7	15.4
	Upper West Side	19.0	65.6	15.4

Shown graphically

```
( (df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100)
| .plot(kind='barh', width=0.9).legend(loc='upper center', ncol = 3, bbox_to_anchor=(0.5, 1.1)) )
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>

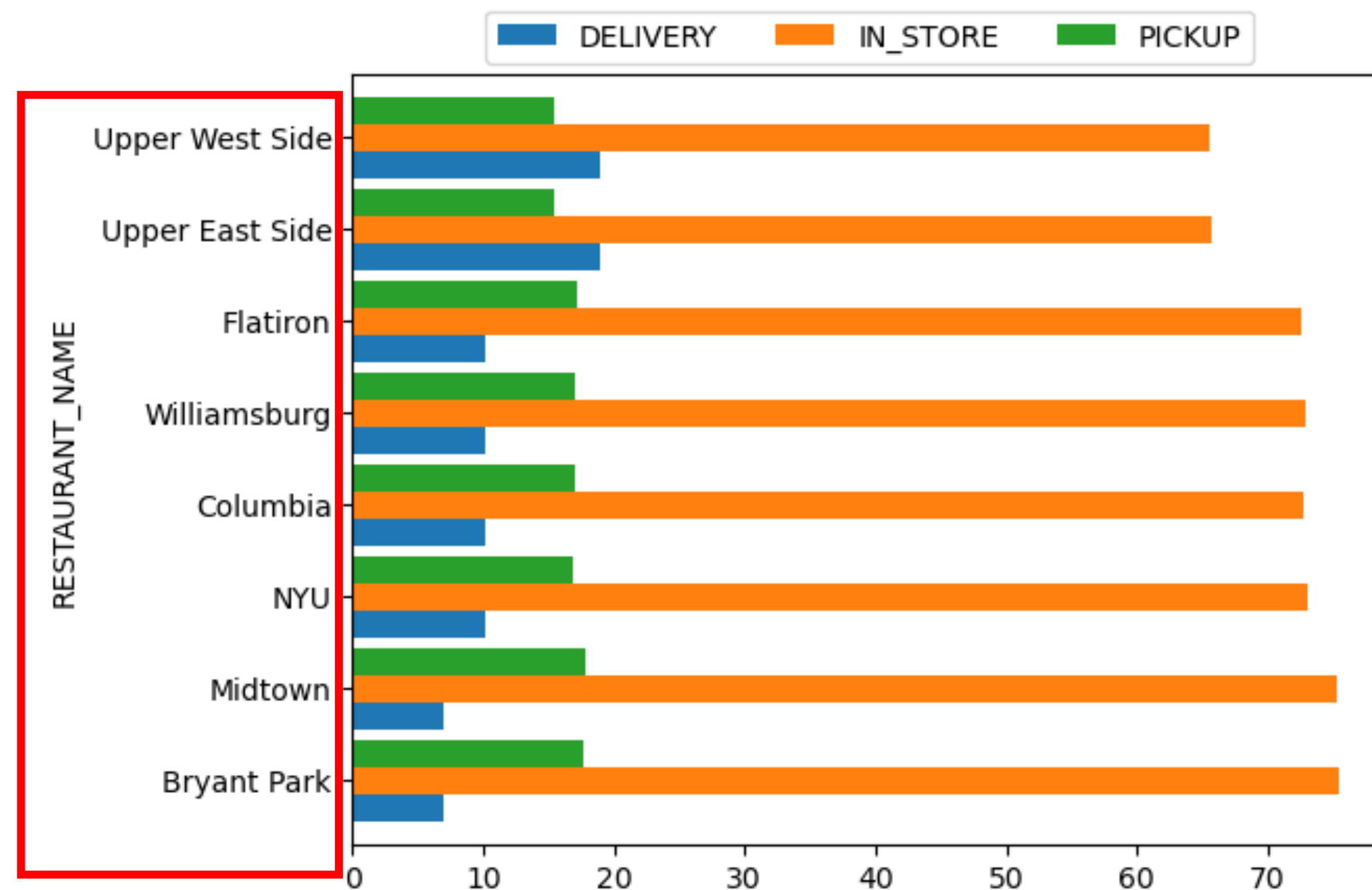


Shown graphically

```
(df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100).plot(kind='bar', width=0.9).legend(loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.1))
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>

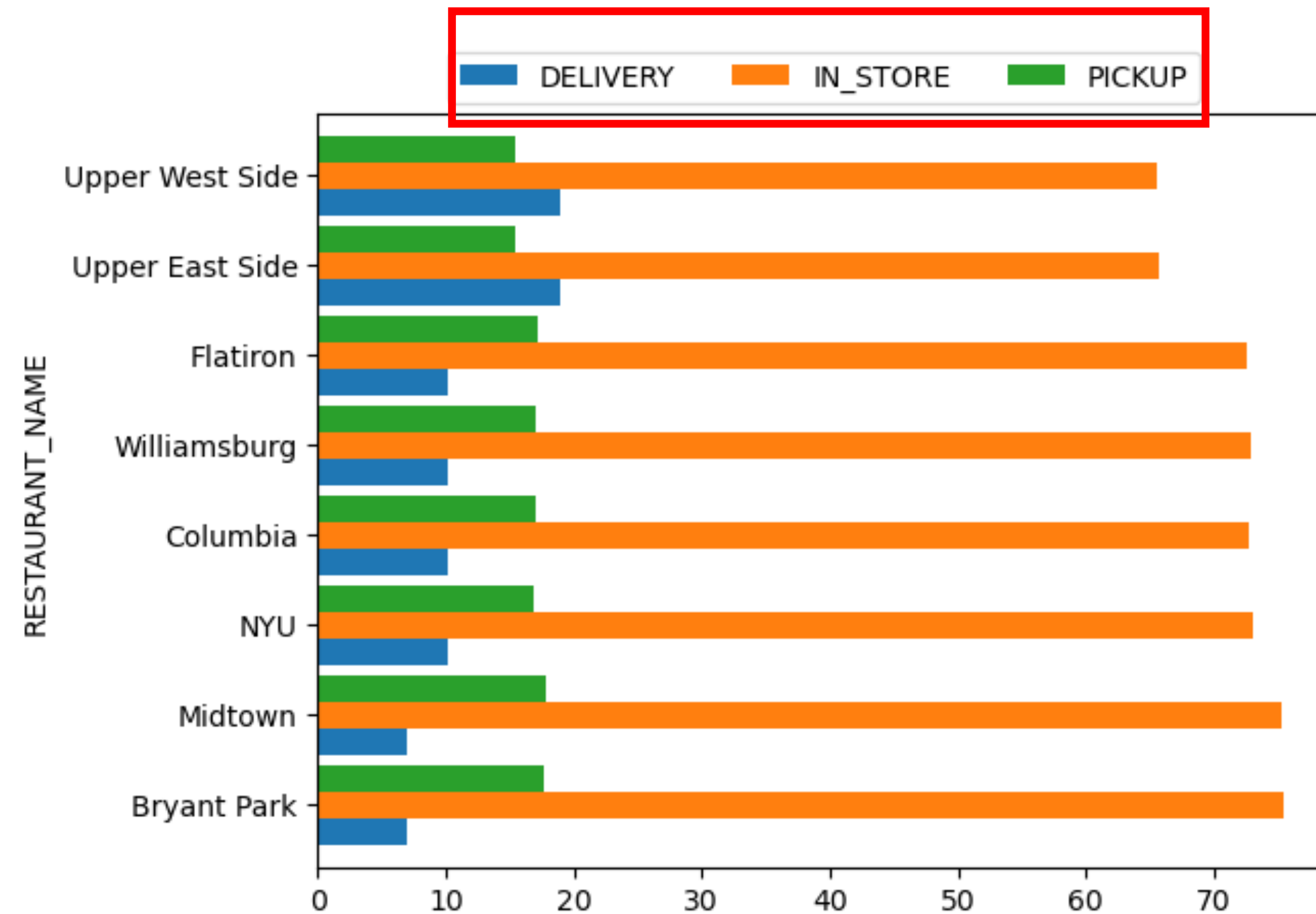


Shown graphically

```
(df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100)
| .plot(kind='barh', width=0.9).legend(loc='upper center', ncol = 3, bbox_to_anchor=(0.5, 1.1)) )
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>



Shown graphically

```
(df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100).plot(kind='barh', width=0.9).legend(loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.1))
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>

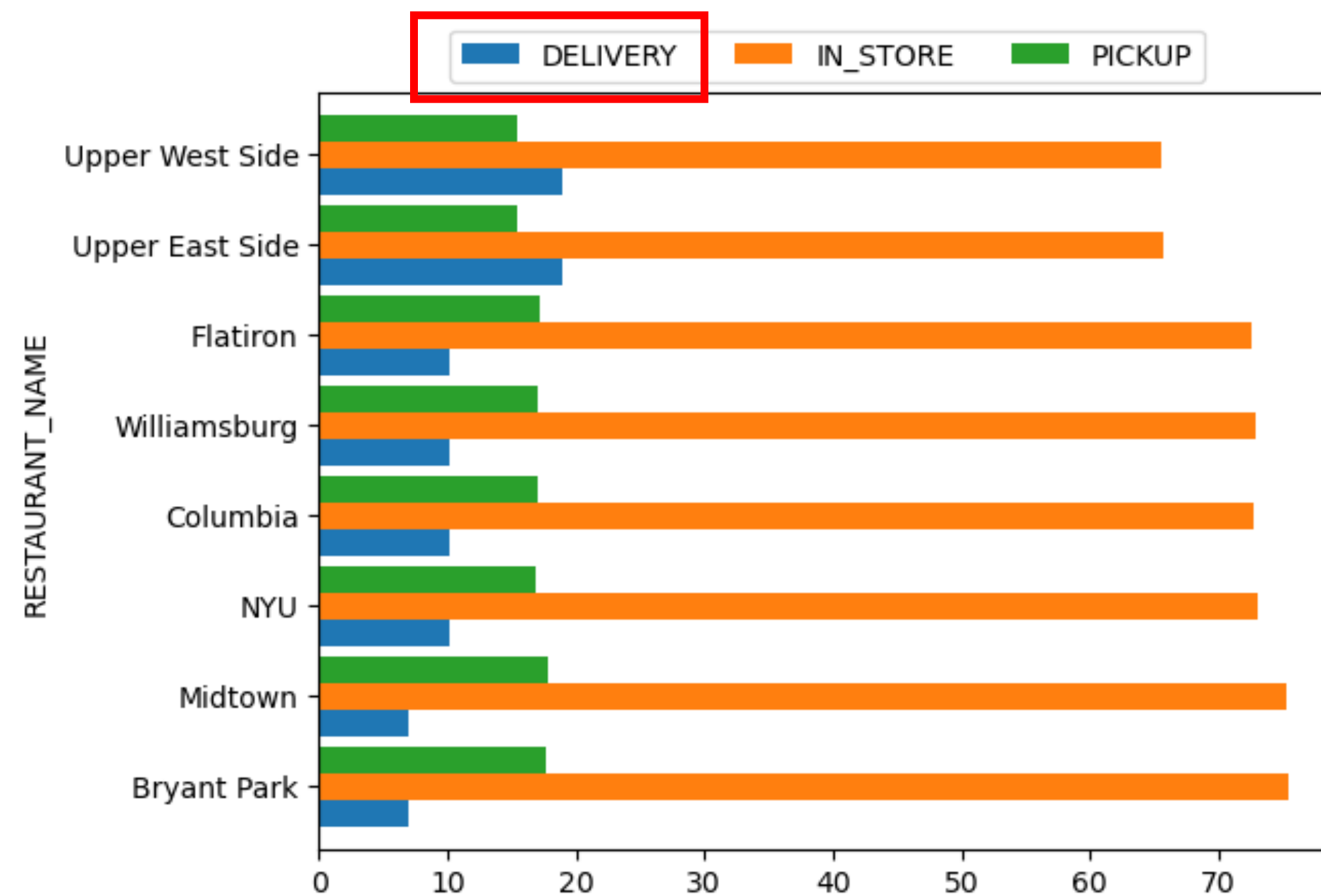


Shown graphically

```
(df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100).plot(kind='barh', width=0.9).legend(loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.1))
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>



Shown graphically

```
(df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100).plot(kind='barh', width=0.9).legend(loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.1))
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15e43b520>

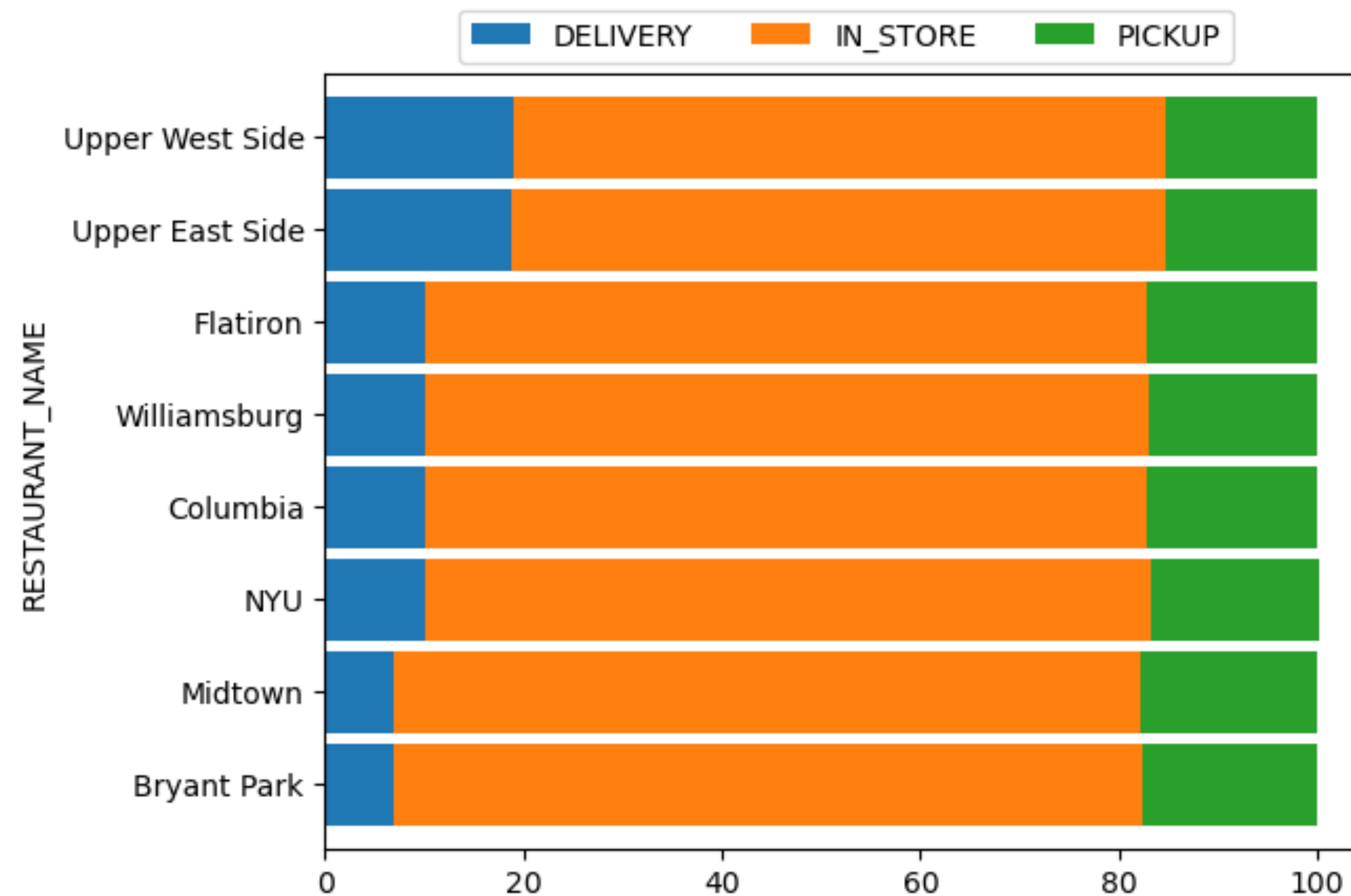


Or we could stack the bars

```
( (df_orders.groupby('RESTAURANT_NAME').TYPE.value_counts(normalize=True).unstack().sort_values('DELIVERY').round(3)*100)
  | .plot(kind='barh', stacked = True, width=0.9).legend(loc='upper center', ncol = 3, bbox_to_anchor=(0.5, 1.1)) )
```

✓ 0.6s

<matplotlib.legend.Legend at 0x15ee0dc00>



Q4: how to staff for success?

Dig needs to release their staffing **schedules in advance**.

How might we use our datasets to facilitate this process of schedule design?

When do orders occur?

```
df_orders.groupby([df_orders.DATETIME.dt.hour, 'RESTAURANT_NAME']).ORDER_ID.count().unstack()
```

✓ 0.3s

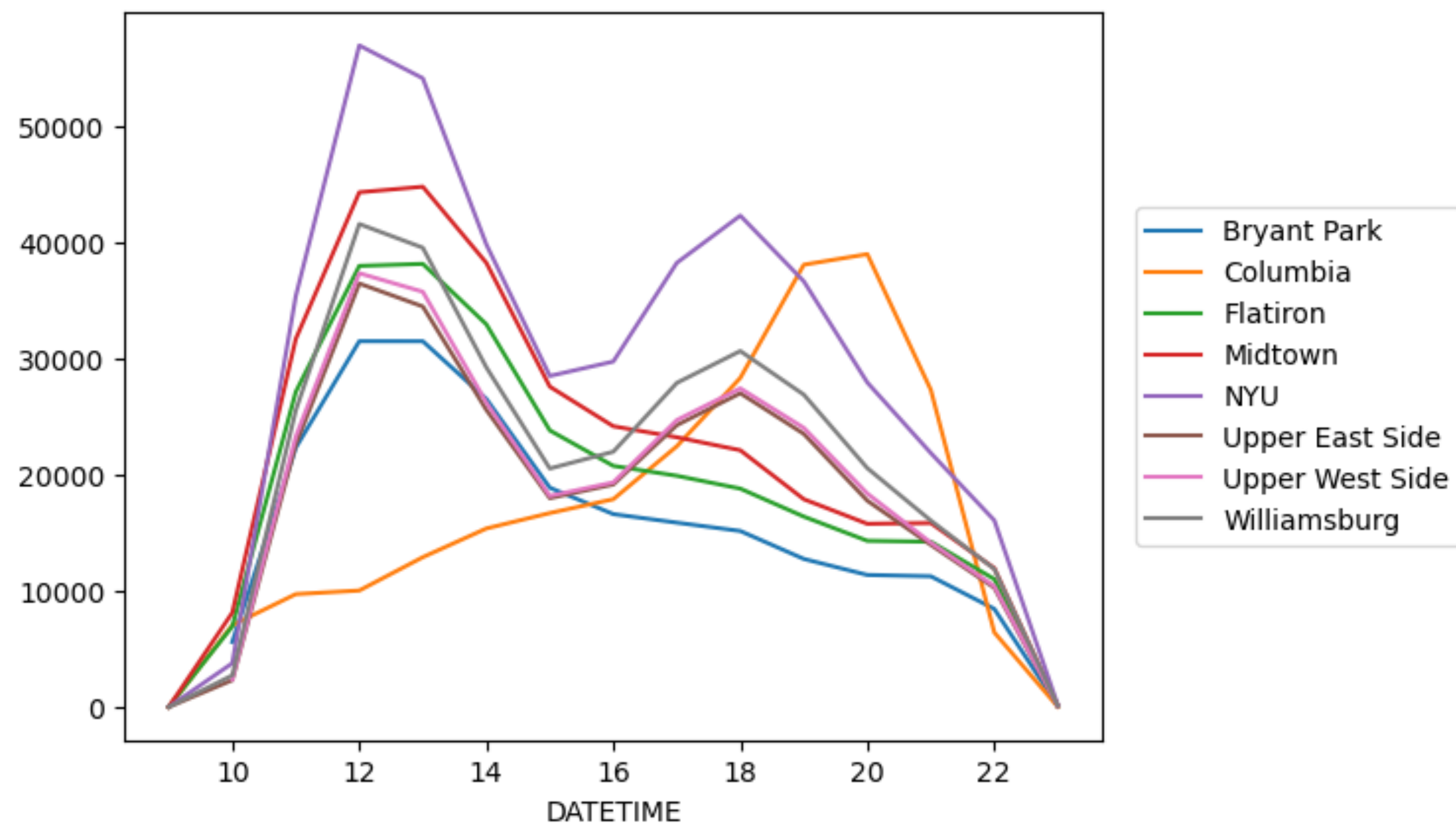
RESTAURANT_NAME	Bryant Park	Columbia	Flatiron	Midtown	NYU	Upper East Side	Upper West Side	Williamsburg
DATETIME								
9	NaN	2.0	2.0	3.0	5.0	1.0	NaN	2.0
10	5581.0	7035.0	6927.0	8106.0	3756.0	2306.0	2374.0	2715.0
11	22289.0	9698.0	27094.0	31643.0	35330.0	22564.0	23237.0	25502.0
12	31444.0	10008.0	37918.0	44244.0	56872.0	36402.0	37286.0	41518.0
13	31442.0	12872.0	38091.0	44724.0	54048.0	34428.0	35695.0	39481.0
14	26484.0	15341.0	32899.0	38184.0	39774.0	25547.0	26172.0	29196.0
15	18848.0	16667.0	23772.0	27537.0	28455.0	17947.0	18141.0	20496.0
16	16577.0	17859.0	20724.0	24132.0	29695.0	19126.0	19303.0	21941.0
17	15844.0	22412.0	19874.0	23185.0	38183.0	24224.0	24658.0	27857.0
18	15126.0	28287.0	18769.0	22068.0	42239.0	26970.0	27386.0	30598.0
19	12724.0	38019.0	16364.0	17859.0	36590.0	23476.0	23990.0	26848.0
20	11347.0	38925.0	14282.0	15745.0	27894.0	17732.0	18345.0	20530.0
21	11233.0	27265.0	14203.0	15813.0	21828.0	13939.0	14124.0	16023.0
22	8428.0	6405.0	10987.0	11925.0	16042.0	10245.0	10379.0	11860.0
23	78.0	15.0	77.0	97.0	149.0	94.0	96.0	107.0

Let's visualize order times!

```
df_orders.groupby([df_orders.DATETIME.dt.hour, 'RESTAURANT_NAME']).ORDER_ID.count().unstack().plot().legend(loc='center', bbox_to_anchor=(1.2, 0.5))
```

✓ 0.4s

<matplotlib.legend.Legend at 0x14e099030>

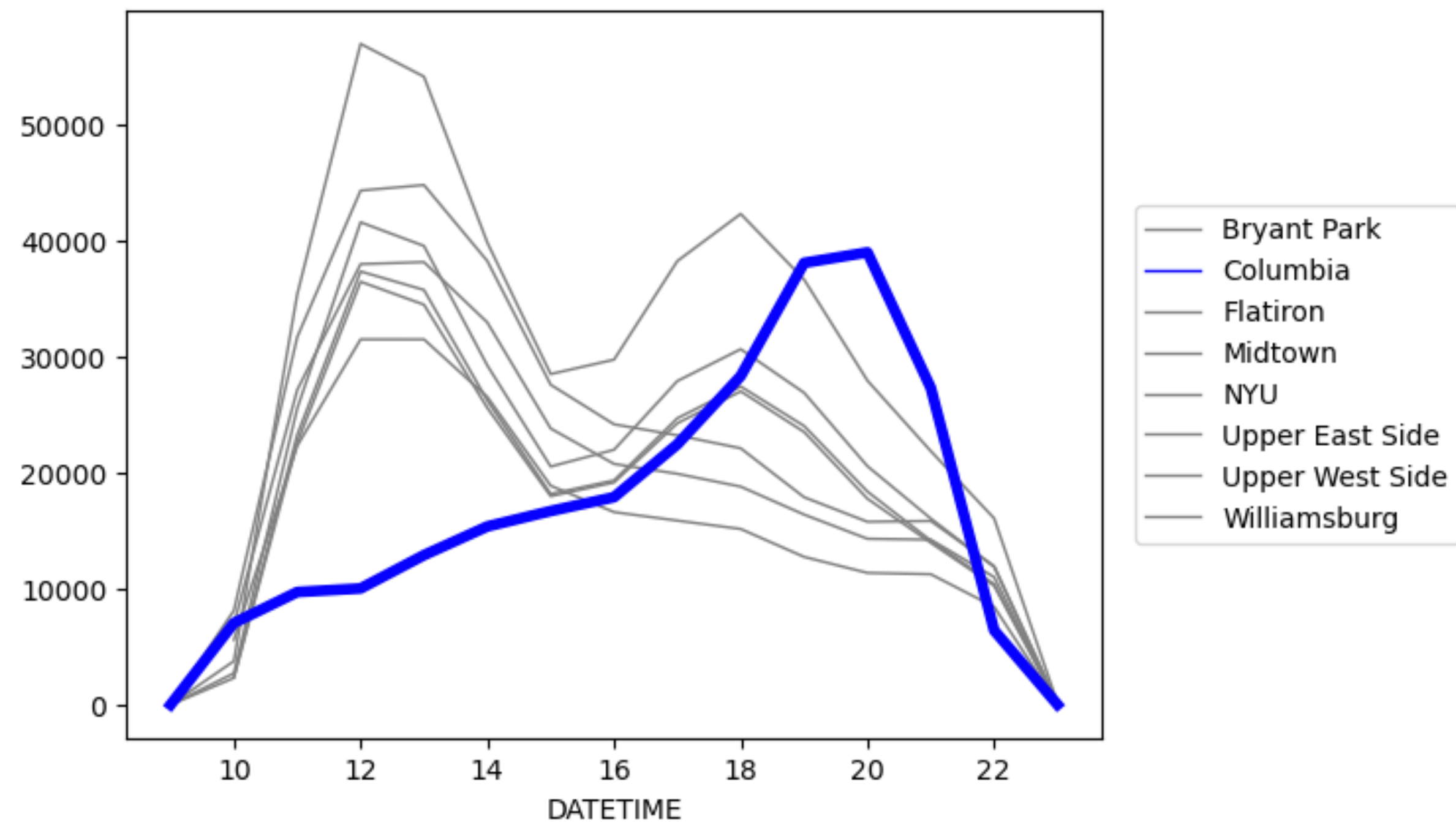


Emphasize a single line

```
1 colors = ['grey', 'blue', 'grey', 'grey', 'grey', 'grey', 'grey', 'grey']
2 df_orders.groupby([df_orders.DATETIME.dt.hour, 'RESTAURANT_NAME']).ORDER_ID.count().unstack().plot(color = colors, linewidth = 1).legend(loc='center', bbox_to_anchor=(1.2, 0.5))
3 plt.plot(df_orders[df_orders.RESTAURANT_NAME == 'Columbia'].groupby([df_orders.DATETIME.dt.hour]).ORDER_ID.count(), color = 'blue', linewidth=4)
```

✓ 1.2s

[<matplotlib.lines.Line2D at 0x17d27dd20>]



Data-driven business decisions

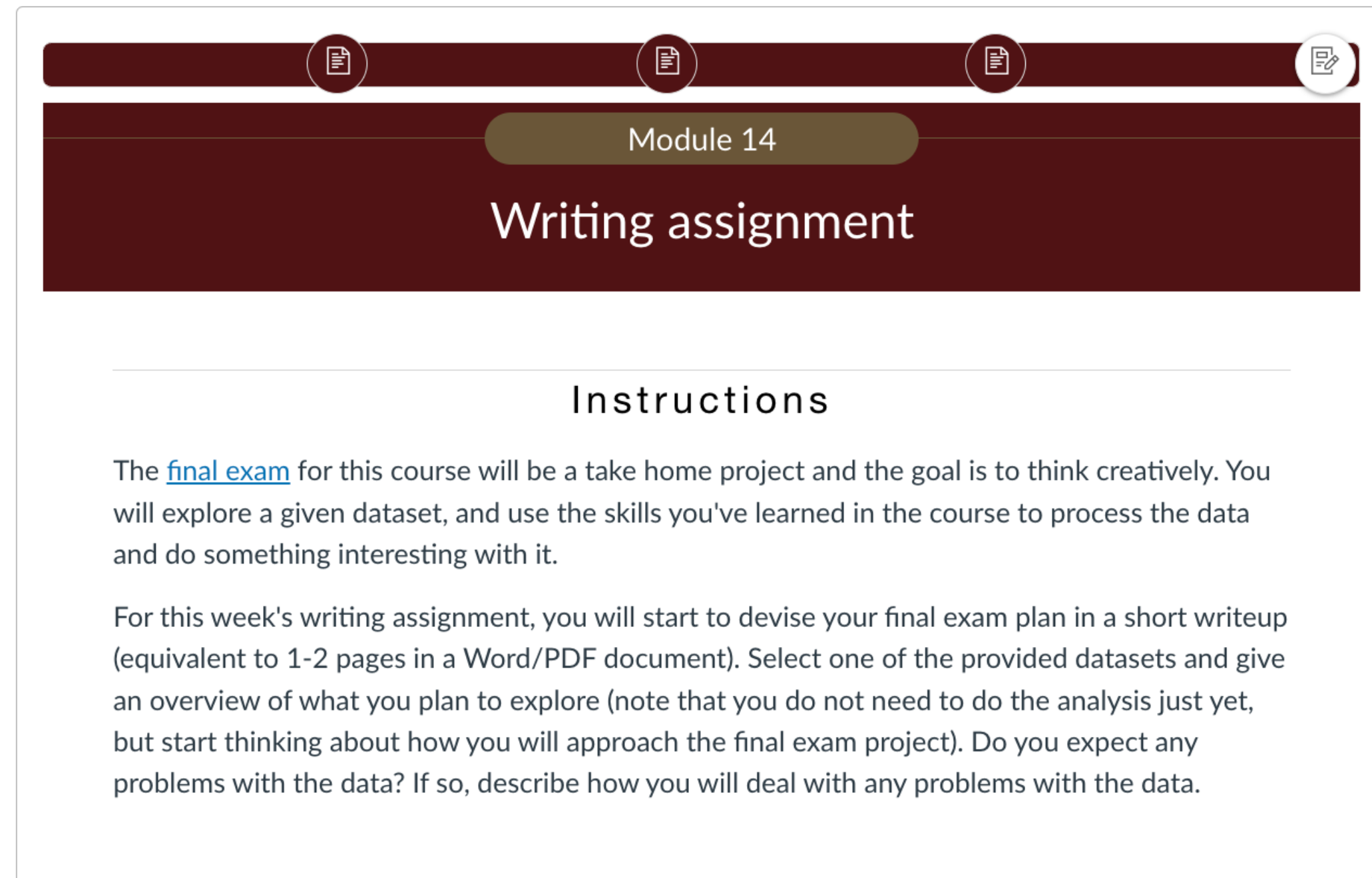
1. Where to launch new drinks?
2. What to include on delivery menu?
3. Where to expand delivery service?
4. How to staff for success?

Final Exam

- Take home project
- Select 1 of 4 available datasets
- Analyze the data to inform a business decision
- Submit code along with a short write up by **May 2 at 10pm**

Next week

Writing assignment due April 25



The screenshot shows a Canvas LMS interface. At the top, there is a dark red header bar with four circular icons containing document symbols. Below this, a dark red banner displays "Module 14" in a light-colored rounded rectangle, followed by "Writing assignment" in white text. The main content area has a light gray background and is titled "Instructions" in bold. The text below reads: "The [final exam](#) for this course will be a take home project and the goal is to think creatively. You will explore a given dataset, and use the skills you've learned in the course to process the data and do something interesting with it." A second paragraph follows: "For this week's writing assignment, you will start to devise your final exam plan in a short writeup (equivalent to 1-2 pages in a Word/PDF document). Select one of the provided datasets and give an overview of what you plan to explore (note that you do not need to do the analysis just yet, but start thinking about how you will approach the final exam project). Do you expect any problems with the data? If so, describe how you will deal with any problems with the data."

Next week: **No lecture**